[Nam Le] Process Fundamental, Memory layout

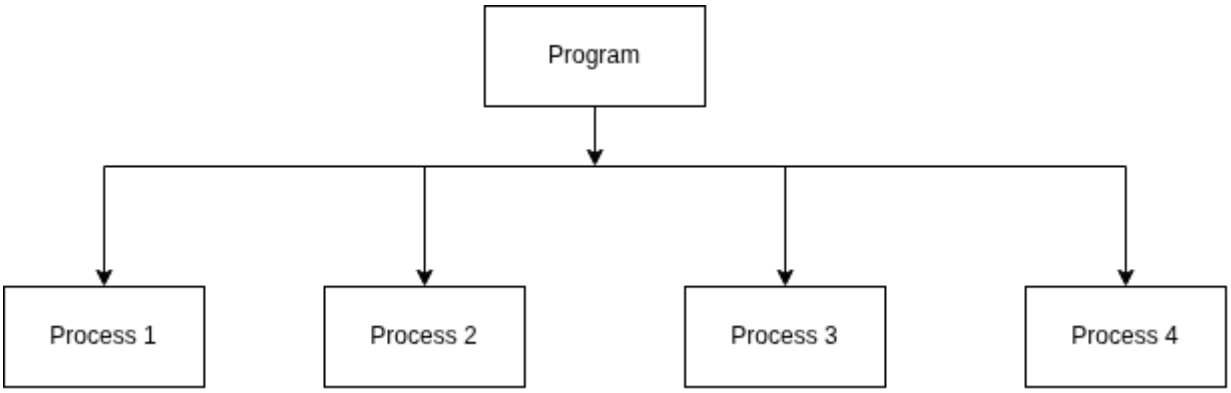
1. Process Fundamental

1.1 Definition

Program: A program is a file containing a range of information that describes how to construct a process at run time

Process: is an abstract entity, defined by the kernel, to which system resources are allocated in order to execute a program.

> One program may be used to construct many processes, or, put conversely, many processes may be running the same program.



1.2 Process ID & Parent Process ID (PID & PPID)

Each process has a **process ID** (PID), a positive integer that uniquely identifies the process on the system

```
#include <unistd.h>
pid_t getpid(void);

/* The pid_t data type used for the return value of getpid() is an integer type specified
by SUSv3 for the purpose of storing process IDs. */
```

> There is no fixed relationship between a program and the process ID of the process that is created to run that program.

Each process has a **parent—the process** that created it. A process can find out the **process ID of its parent** using the getppid() system call.

```
#include <unistd.h>
pid_t getppid(void);
```

1.3 Process Creation

1.3.1 fork() & vfork()

fork()

The **fork()** system call allows one process, the parent, to create a new process, the child. This is done by making the new child process an (almost) exact duplicate of the parent: the child obtains copies of the parent's stack, data, heap, and text segments

EXAMPLE USING FORK

Lab 1.1

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
static int idata = 111;                       /* Allocated in data segment */
int main()
{
    int istack = 222;                         /* Allocated in stack segment */
    pid_t childPid;

    switch (childPid = fork()) {
    case -1:
        exit(1);

    case 0:
        idata *= 3;
        istack *= 3;
        break;

    default:
        sleep(3);                           /* Give child a chance to execute */
        wait(NULL);
        break;
}

/* Both parent and child come here */

printf("PID=%ld %s idata=%d istack=%d\n", (long) getpid(),
(childPid == 0) ? "(child) " : "(parent)", idata, istack);

exit(0);
}
```

OUTPUT:

```
$ ./t_fork
PID=28557 (child) idata=333 istack=666           /* Then wait 3 seconds to child have a chance
to execute */
```

```
PID=28556 (parent) idata=111 istack=222
```

We have this result because with fork() function only **call 1 time**, but return **2 different results** for 2 processes

> Return 0 **in successfully created child**
> Return **PID** of child process inside parent process

vfork()

Like fork(), `vfork()` is used by the calling process to create a new child process. But child is designed to be use memory of parent and force parent wait the child perform an immediate `exec()` or `_exit()` call

fork()	vfork()
Create child with virtual address space	Child share memory to parent
Modern Linux use Copy-on-Write	No copy page/page table
Parent & child can run in parallel	Parent must be suspend
Child fix variable --> Parent cant see	Child fix variable → parent sees the change
Continue to code normally	Child must be activity restrictions
Use much	Rarely use

Example:

```
int x = 10;

pid_t pid = vfork();

if (pid == 0) {
    x = 20;
    _exit(0);
}

printf("%d\n", x);
```

Với `fork()` :

```
Parent: x = 10
Child: x = 10
```

```
Child fix x = 20

Parent still see x = 10
```

Với `vfork():`

```
Parent: x = 10
        |
        |--- Child use same memory
        |
        |--- Child fix x = 20
                      |
                      ▼
                      Parent continue with x = 20
```

Specially: parent is blocked
Với `fork()` :

```
             fork()
            /      \
        Parent     Child
          |          |
       Can be       Can be
      still run    still run
```

Với `vfork():`

```
              vfork()
                |
                ▼
              Child run
                |
        ┌───────┴────────┐
        |                |
     exec()           _exit()
        |                |
        └───────┬────────┘
                ▼
      Parent continue to run
```

1.3.2 exit() & _exit()

exit()	_exit()
Library function of C (stdlib.h)	**System call** (unistd.h)
Do **clean up** before end	**Immediately end**
Call functions `atexit()` / `on_exit()`	Dont call
Flush stdio buffers (printf, fopen ...)	Dont flush stdio buffers
Call `_exit()` at the end	End directly process

Or: `exit()` = **clean up --> flush buffer --> `_exit()` --> process die**

Example1:

```
printf("Hello");
exit(0);
```

Output: `Hello`

`Hello can be flush to output before process end`

Example 2:

```
printf("Hello");
_exit(0);
```

Output:
Hello can not be appear if it's `stdio` buffer.

Different from `return` in **main**:

Feature	`return` from main function	`exit`
Where can use?	Only inside **main** function .	Everywhere in program.
Local variable in main function	free exactly (call Destructor).	No free.
Global variable	Free.	Free.
Delete stdio buffer (Flush)	Yes.	Yes.

1.4 Waiting on Child Process

1.4.1 wait() System call

The `wait()` system call wait for one of the children of the calling process to terminate and returns the termination status of that child in the buffer pointed to by `status`

```
#include <sys/wait.h>
pid_t wait(int *status);  /* Returns process ID of terminated child, or –1 on error */
```

In summary, the `wait()` system call does:

1. If no child of the calling process has yet terminated. the call block untils one of the children terminates. If a child has already terminated by the time of the call, `wait()` return immediately.
2. If `status` is not NULL, the information about how the child terminated is returned in the integer to which `status` points
3. Kernel adds the process CPU times and resource usage statistics to running totals for all children of this parent process.
4. As its function result, wait() returns the process ID of the child that has terminated.

On error, wait() returns –1. One possible error is that the calling process has no (previously unwaited-for) children, which is indicated by the errno value ECHILD. This means that we can use the following loop to wait for all children of the calling pro-cess to terminate:

```
while ((childPid = wait(NULL)) != -1)
continue;
if (errno != ECHILD)     /* An unexpected error... */
errExit("wait");
```

The wait status integer populated by `wait()` can be dissected using the same standard family of macros as `waitpid()`. For example:

- **`WIFEXITED(status)`**: Evaluates to true if the child exited normally (e.g., via `exit()` or `_exit()`), and **`WEXITSTATUS(status)`** extracts the child's return code .
- **`WIFSIGNALED(status)`**: Evaluates to true if the child was terminated abruptly by a signal, and **`WTERMSIG(status)`** returns the signal number that caused the termination .

Lab 1.2

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
```

```
int main() {
    pid_t childPid;
    int status;

    switch (childPid = fork()) {
        case -1:
            perror("fork failed");
            exit(EXIT_FAILURE);

        case 0:
            printf("[Child] PID is %ld. Work in 2 seconds...\n", (long) getpid());
            sleep(2);
            printf("[Child] Finish work. Out with exit code 12.\n");
            exit(12);

        default:
            printf("[Parent] PID is %ld. Calling wait() to wait for child...\n", (long)
getpid());

            pid_t reapedPid = wait(&status);

            if (reapedPid == -1) {
                perror("wait failed");
                exit(EXIT_FAILURE);
            }

            printf("[Parent] reaped (clean) child process have PID: %ld\n", (long) reapedPid);

            if (WIFEXITED(status)) {
                printf("[Parent] Child out normally. The exit code of child is: %d\n",
WEXITSTATUS(status));
            } else if (WIFSIGNALED(status)) {
                printf("[Parent] Child is killed by signal, number is: %d\n",
WTERMSIG(status));
            }
            break;
    }

    return 0;
}
```

Run: `./test_wait_function`

Output:

```
[Parent] PID is 73052. Calling wait() to wait for child...
[Child] PID là 73053. Work in 2 seconds...
[Child] Finish work. Out with exit code 12.
[Parent] reaped (clean) child process have PID: 73053
[Parent] Child out normally. The exit code of child is: 12
```

1.4.2 The waitpid() System call

The `waitpid()` system call is designed to monitor state changes in child processes

Function Prototype:

```
#include <sys/wait.h>
pid_t waitpid(pid_t pid, int *status, int options);  /* Returns process ID of child, 0 (see
text),
                                                        or –1 on   error */
```

1. **The pid Argument**
   Unlike `wait()`, which reaps any terminated child, the `pid` argument in `waitpid()` lets you select precisely which child (or groups of children) to wait for

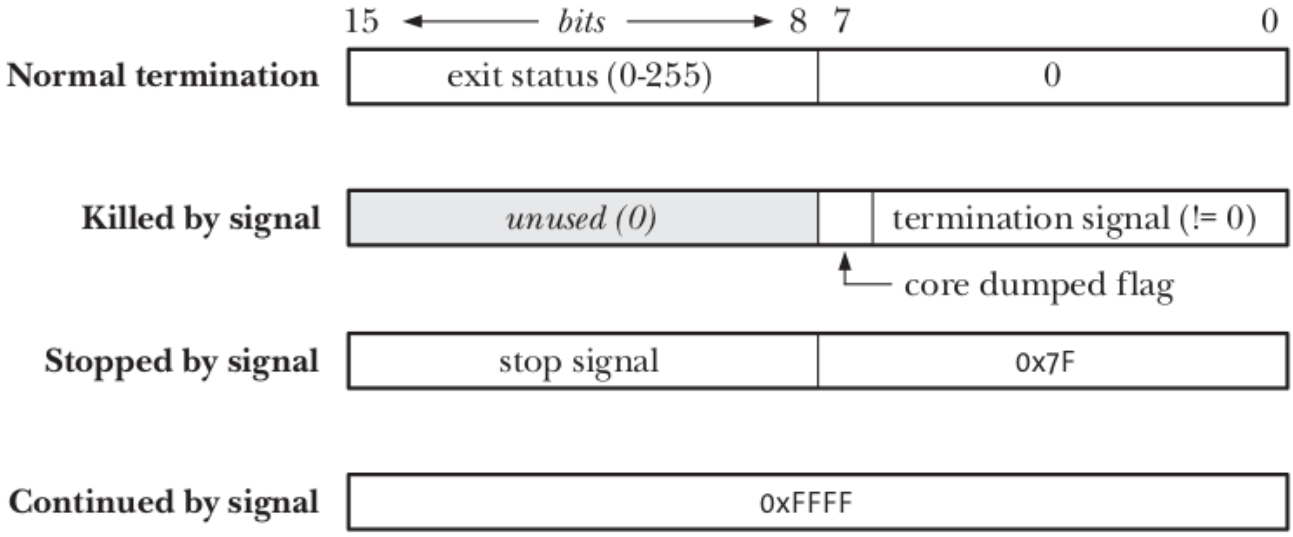
- **pid** > 0: Waits for the specific child whose process ID is exactly equal to pid
- **pid** == 0: Waits for any child process in the same process group as the calling parent
- **pid** < -1: Waits for any child process whose process group ID is equal to the absolute value of pid
- **pid** == -1: Waits for any child process. The call wait(&status) is completely equivalent to waitpid(-1, &status, 0)

2. **The options Argument (Controlling Behavior)**
   The `options` argument is a bit mask (created by bitwise ORing | flags) that alters how ``waitpid()``` behaves

- **WNOHANG**: Performs a **non-blocking check (poll)**. If no child specified by `pid` has changed state yet, `waitpid()` does not block; it immediately returns **0**. This allows parent processes to continuously perform other background work while periodically checking on children
- **WUNTRACED**: Instructs the system call to also return status information if a child is **stopped by a signal** (such as SIGSTOP), rather than only reporting terminated children

1.4.3 The Wait Status Value

The status value returned by wait() and waitpid() allows us to distinguish the following events for the child:



1. The child terminated by calling _exit() (or exit()), specifying an integer exit status
2. The child was terminated by the delivery of an unhandled signal.
3. The child was stopped by a signal, and waitpid() was called with the WUNTRACED flag.
4. The child was resumed by a SIGCONT signal, and waitpid() was called with the WCONTINUED flag.

MACRO STANDARD for encode state:

- **NORMAL Termination**
  - **WIFEXITED(status)**: Return `true` if **child** end normally
  - **WEXITSTATUS(status)**: print reality out code of child process (in 8 high bit)
- **Killed by Signal**
  - **WIFSIGNALED(status)**: Return `true` if **child** process killed by a signal
  - **WTERMSIG(status)**: Return the number of signal killed **child** signal
  - **WCOREDUMP(status)**: Return `true` if child process is killed and system created a **core dump file** for debug.
- **STOPPED**
  - **WIFSTOPPED(status)**: Return `true` if **child** is stopped by a signal.
  - **WSTOPSIG(status)**: Return the number of signal stopped child process.
- **Continued**
  - **WIFCONTINUED(status):** Return `true` if child process is stopped but run again by receive **SIGCONT** signal

1.4.4 The waitid() System call

Syntax

```
#include <sys/wait.h>
int waitid(idtype_t idtype, id_t id, siginfo_t *infop, int options);          /* Returns 0 on
success
                    or if WNOHANG was specified and there were no children to wait for, or –1
on error */
```

Specific child processes wait for (idtype and id)
argv `idtype` and `id` operate together to verify the **chill process** be monitored

- **idtype == P_ALL**: wait any child process, `id` is ignored
- **idtype == P_PID**: wait child process which have **PID** = `id`
- **idtype == P_PGID**: wait any child process in **Process Group ID** equal to `id`

NOTE: Unlike `waitpid()`, you can't transmit `id = 0` to specific **present group process of parent process**. If we want to wait to our group process, we must call **getpgrp()** to get PGID of parent then push (transmit) it into id

Control Specifically event by `option`
waitid() allows parent process to choose exactly state events they want to catch, by combine these following **flags**:

- **WEXCITED**: Wait for end of child process (either normally or killed by signal)
- **WSTOPPED**: Wait for chill process is stopped by a signal
- **WCONTINUED**: Wait for previous child process was stopped but continue run again by `SIGCONT`

Additional action flag:

- **WNOHANG**: run in **non-blocking** mode. If dont have any child process changes the stage, return 0 immediately.
- **WNOWAIT**: allow get state information of child but **can't consumed** it. Child process still remain old state for parent process can run another `wait()` to get this information later.

Extract information in detail through `signinfo_t`
When event sussesfully, `waitid()` doesn't return simplely integer `status` like `waitpid()`. Instead of it, it writes all detail information in **siginfo_t** pointed by `*infop`

- **si_pid**: PID of child process state is changed
- **si_signo**: Always set **SIGCHLD**
- **si_uid**: Real User ID of child process (LINUX support, other no support)
- **si_code**: Specific the detail event occur with child process
  - **CLD_EXITED**: Child end by call _exit()
  - **CLD_KILLED**: Child is killed by a signal.
  - **CLD_STOPPED**: Child is stopped by a signal

    - **CLD_CONTINUED**: Child continue to run by **SIGNCONT**
- **si_status**: Contain the exit code of child (if si_code is CLD_EXITED) or signal number affects on child

FAST COMPARE

Feature	wait()	waitpid()	waitid()
wait object	**any** child	**specifical** child (PID), (PGID) or any child	**flexible** through `idtype` (P_PID, P_PGID, P_ALL) and ID
non-blocking	**NO** (Always suspend parent until child out)	**YES** (use **WNOHANG** in `option`)	**YES** (use **WNOHANG** in `option`)
check event	only know when child terminate	Catch child event **end, stop (WUNTRACED)** or **continue (WCONTINUED)**	control detail through independence flag: **WEXICTED, WSTOPPED, WCONTINUE**
infor return	**PID** of child and simplified **status** integer	**PID** of child and simplified **status** integer	Return **siginfo_t** structure in detail (PID, UID, event code **si_code**, out code **si_status**)

1.5 Program execution

1.5.1 Executing a New Program: execve()

1. Core Mechanism

- **Replace the segment memory**: Whenever call, `execve()` loads a new program into current process's memory. The old program is discarded. The process's stack, data, and heap are replaced by those of the new program.
- **Start executing**: After executing various C library run-time startup code and program initialization code, the new program commences execution at its main() function.
- **Remain PID**: After `execve()`, PID still remain because it doesn't create a new process, old process self-replace source and data to new source and data

2. Prototype & Arguments

```
#include <unistd.h>
int execve(const char *pathname, char *const argv[], char *const envp[]);   /* Never returns
on success;
                                                                              returns –1 on
```

```
error */
```

Arguments:

- **pathname**: path to new execution program. This pathname can be absolute (start with /) or relative to the current working directory
- **argv**: specifies the command-line arguments to be passed to the new program (argv argument to a C main() function); it is a NULL-terminated list of pointers to character strings. Typically, this value is the same as the basename (i.e., the final component) of pathname
- **envp**: specifies the environment list for the new program. Same `argv`, including string pointer `name=value` and end by NULL pointer

Lab 1.3

3. Example

```
  #include <stdio.h>
#include <unistd.h>

int main(void)
{
    char *args[] = {"ls", "-l", NULL};
    char *envp[] = {NULL};

    execve("/bin/ls", args, envp);
    printf("Hello");                      // If execve sucess, not print Hello
    // Only run after execve() unsucess
    perror("execve");
    return 1;
}
```

Compile: `./execve_demo`

Result:

```
total 48
drwxrwxr-x 2 hoainam0508 hoainam0508  4096 Aug 20 15:45 compilation
-rwxrwxr-x 1 hoainam0508 hoainam0508 16104 Aug 24 09:29 execve
-rw-rw-r-- 1 hoainam0508 hoainam0508   315 Aug 24 09:29 execve_demo.c
-rwxrwxr-x 1 hoainam0508 hoainam0508 16248 Aug 20 15:43 function_pointer
-rw-rw-r-- 1 hoainam0508 hoainam0508   417 Aug 20 15:43 function_pointer.c
-rw-rw-r-- 1 hoainam0508 hoainam0508  2163 Aug 20 13:37 linked_list.c
```

Understand that: **execve()** dont create new process, It replaces **current process image** by **/bin/ls** & run the command `ls -l`

```
exec_demo
   |
   | execve("/bin/ls", ...)
   ↓
 /bin/ls
```

So that: PID before execve() = PID after execve();
Note that: `Hello` is only printed if execve unsucess

1.5.2 exec() Library function

1. Syntax

```
#include <unistd.h>
int execle(const char *pathname, const char *arg, ...
/* , (char *) NULL, char *const envp[] */ );
int execlp(const char *filename, const char *arg, ...
/* , (char *) NULL */);
int execvp(const char *filename, char *const argv[]);
int execv(const char *pathname, char *const argv[]);
int execl(const char *pathname, const char *arg, ...
/* , (char *) NULL */);


                                    None of the above returns on success; all return –1 on
error
```

All of these functions are layered on top of execve(), and they differ from one another and from execve() only in the way in which the program name, argument list, and environment of the new program are specified.

Trick:

```
l → list
v → vector (array)
p → PATH search
e → environment specific
```

Function	Program	Arguments	PATH search	Environment

`execl`	pathname	list	□	caller
`execv`	pathname	array	□	caller
`execlp`	filename	list	□	caller
`execvp`	filename	array	□	caller
`execle`	pathname	list	□	**custom** `envp`
`execve`	pathname	array	□	**custom** `envp`

2. Example

2.1 **execl()** `execl("/bin/ls","ls","-l","-a",NULL);`
Same as: `/bin/ls -l -a`

2.2 **execv()** `char *argv[] = {"ls","-l","-a",NULL};`
Same as: `/bin/ls -l -a`

2.3 **execlp()**: `execlp("ls", "ls", "-l", NULL);`
"ls" doesn't contain `/`, execlp() will use `PATH`

Ex: My program path is
`echo $PATH`
Result:

```
/home/hoainam0508/miniconda3/bin:
/home/hoainam0508/.config/Code/User/globalStorage/github.copilotchat/debugCommand:
/home/hoainam0508/.config/Code/User/globalStorage/github.copilotchat/copilotCli:
/home/hoainam0508/miniconda3/bin:
/home/hoainam0508/.nvm/versions/node/v20.20.2/bin:
/usr/local/sbin:/usr/local/bin:
/usr/sbin:
/usr/bin:
/sbin:
/bin:
/usr/games:
/usr/local/games:
/snap/bin:
/snap/bin
```

Program found `/bin/ls`
So, the example same as: `ls -l`

2.4 **execvp()** replace list to array

2.5 **execle()**

`char* envp[] = {"MY_NAME=Nam", "MY_OS=Linux",NULL};`

```
execle(
    "/usr/bin/env",
    "env",
    NULL,
    envp
);
```

Other functions will use current environment of process/caller
Ex: export `NAME = nam` in command shell
Command shell to check environment of process: `env` or `printenv`

The result look like:

```
SHELL=/bin/bash
SESSION_MANAGER=local/hoainam0508-ThinkPad-T14s-Gen-4:@/tmp/.ICE-unix/1802,unix/hoainam0508-
ThinkPad-T14s-Gen-4:/tmp/.ICE-unix/1802
QT_ACCESSIBILITY=1
COLORTERM=truecolor
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
SSH_AGENT_LAUNCHER=gnome-keyring
NVM_INC=/home/hoainam0508/.nvm/versions/node/v20.20.2/include/node
XDG_MENU_PREFIX=gnome-
............
HOME=/home/hoainam0508
USERNAME=hoainam0508
............
INVOCATION_ID=e2172e5835404968a762ec115fa97112
MANAGERPID=1653
CHROME_DESKTOP=code.desktop
...............
TERM_PROGRAM=vscode
_=/usr/bin/printenv
```

1.6 System function

system() will do list of work, like:

```
system("command")
       |
       ├── fork()
       |
       ↓
    Child
       |
       └── execl("/bin/sh", "sh", "-c", command, NULL)
                        |
                        ↓
                   command
       |
       ↓
    Parent
       |
       └── wait()
```

Example:

```
system("ls -l");
```

It will same with

```
pid_t pid = fork();

if (pid == 0) {
    execl("/bin/sh", "sh", "-c", "ls -l", NULL);
    _exit(127);
}
waitpid(pid, NULL, 0);
```

2. Process Relation && Job Control

2.1 Zombie

Definition : Occurs when child that terminates before its parent has had a chance to perform a wait()
Mechanism:

- Allow parent can check **end state** of chill anytime, OS will not delele completely child. Instead, it will be free all source of chill and convert it to **zombie**
- an entry for this zombie will be maintained indefinitely(!) in the **kernel's process table** to can see **PID, end state, resource is used**

undying: a zombie process can't be killed by a signal, not even the (silver bullet) SIGKILL. This ensures that the parent can always eventually perform a wait() to get child data./ **the way to free zombie**:

- whenever parent call `wait()`, OS kernel will be completely delete zombie out of process
- If parent out before call `wait()`, **init process** will adopt this zombie and automatively call `wait()` to free it.

consequence: If parent process create many children but forget call `wait()`, zombie will exist infinitely and fill all table process, system can create new process so far. We must kill parent process to clean it

Lab 1.4

EXAMPLE

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        exit(1);
    }

    if (pid == 0) {
```

```
        printf("Child: PID = %d\n", getpid());              /*55408*/
        exit(0);
    }

    printf("Parent: PID = %d, Child PID = %d\n",             /* 55407 & 55408 */
           getpid(), pid);

    sleep(30);

    return 0;
}
```

Result:

```
hoainam+   55407   20235  0 13:17 pts/3    00:00:00 ./zombie
hoainam+   55408   55407  0 13:17 pts/3    00:00:00 [zombie] <defunct>
```

> While sleep 30s, we can see the 55408 id process is the zombie

2.2 Orphans

Definition: Occurs when parent end before child end
Mechanism: In most cases, the orphan process is reparented to the init process (usually systemd) with PID 1

Lab 1.5

EXAMPLE:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid = fork();
```

```
    if (pid < 0) {
        perror("fork");
        exit(1);
    }

    if (pid == 0) {
        printf("Child: PID = %d, Parent ID = %d\n", getpid(), getppid());
        sleep(10);
        printf("Child after Parent exits: PID = %d, PPID = %d\n", getpid(),
getppid());
        exit(0);
    }

    printf("Parent: PID = %d, Child PID = %d\n", getpid(), pid);

    exit(0);

    return 0;
}
```

Result

```
Parent: PID = 65636, Child PID = 65637
Child: PID = 65637, Parent ID = 65636
/* After 10 seconds */
Child after Parent exits: PID = 65637, PPID = 1653
```

We can see new `PPID = 1653`, not `1` like theory
Since version 3.4, Linux has had the concept of a subreaper. Its mean child can allow another process have ability like `PID 1`. In the example, its `PID 1653`

2.3 Race Condition

A race condition is a situation where the result produced by two processes operating on shared resources depends in an unexpected way on the relative order in which the processes gain access to the CPU(s).

```
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <unistd.h>
#include <sys/mman.h>
#include <sys/wait.h>
int main(void)
{
    int *counter = mmap(
        NULL,
        sizeof(int),
        PROT_READ | PROT_WRITE,
        MAP_SHARED | MAP_ANONYMOUS,
        -1,
        0
    );

    *counter = 0;

    __pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        exit(1);
    }

    for (int i = 0; i < 100000; i++) {
        (*counter)++;
    }

    if (pid == 0) {
        printf("Child done \n");
    } else {
        printf("Parent done\n");
        wait(NULL);
        printf("Final counter = %d\n", *counter);
    }
    return 0;
}
```

RESULT Expect:

```
Parent: +100000
Child:  +100000
----------------
Expected = 200000
```

Reality: `Final counter = 157823`

We can assume this situation:

```
Parent                  Child


Read 100
                        Read 100
Add 1
                        Add 1
Write 101
                        Write 101
```

But expectation is: 100 --> 101 --> 102

Each linux version have **scheduler** will regulate different order process run first

after `fork()`

```
              fork()
              /  \
             /    \
        Parent    Child
           |        |
           |        |
       scheduler quy□t định
           |        |
```

Lab 2

A race condition occurs when coordination between the parent and child processes is not synchronized; for example, the parent execs a program while the child is still using a shared resource or a file descriptor that the parent modified or closed too early.

Example:

```
int pipefd[2];
pipe(pipefd);

pid_t pid = fork();

if (pid == 0) {
    // Child
    sleep(1);

    char buf[100];
    read(pipefd[0], buf, sizeof(buf));

    printf("Child received: %s\n", buf);
} else {
    // Parent
    close(pipefd[0]);

    char *args[] = {"echo", "Hello", NULL};

    execve("/bin/echo", args, NULL);
}
```

2.4 Process Group / Session, Controlling Terminal, Job Control

2.4.1 Process Group

- **Definition**: A process group is a set of one or more processes sharing the same process *group identifier* (PGID). *A new process inherits its parent's process group ID*
- A process group has a **process group leader** , which is the process that creates the group and whose process ID *(PID)* **becomes** the process group ID *(PGID)* of the group. *A process group has a **lifetime**, which is the period of time beginning when the leader creates the group and ending when the last member process leaves the group*
- **API()**:
  - getpgrp(): Get PGID of current process
  - setpgrp(): Set or Replace group process of **PID** to **PGID NOTE**: cannot change PGID of child process if called `exec()`

2.4.2 Sessions

- **definition**: is a collection of process groups
- A process’s **session membership** isdetermined by its **session identifier** `SID`, inherit from parent's process session
- Session Leader: call **`setsid()`** to create new session, **PID** of it become to **SID** of session

2.4.3 Controlling Terminal

One session can connect maximum one **controlling terminal**

- **Establishment**: The controlling terminal is established when the session leader first opens a terminal device. By that, session membership becomes **controlling process** of this terminal

We can run `tty` (Teletypewriter) to see the shell which terminal is using
Result: `/dev/pts/0`

2.4.4 Foreground and Background Process Group

Within a session, only one process can be in the foreground at a particular moment; all of the other process groups in the session are background process groups
Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        exit(1);
    }
```

```
    if (pid == 0) {
        exit(0);
    }

    sleep(10);
    wait(NULL);
    return 0;
}
```

Case 1: Run `./zombie`

```
Parent: PID = 108752, Child PID = 108753
Child: PID = 108753
/* Still wait 10s */
```

It means we are running foreground process group

Case 2: Run `./zombie &`

```
[1] 108752
Parent: PID = 108752, Child PID = 108753
Child: PID = 108753
/* End immediately */
```

It means we are running background process group

Lab 3

Check it by `ps`:

```
ps -o pid,ppid,pgid,sid,tty,stat,cmd
```

```
  PID    PPID    PGID     SID   TT       STAT   CMD
 20235   11442   20235   20235 pts/3    Ss+  /usr/bin/bash --init-file /snap/co
 108753  108752  108752   20235 pts/3    Z+   [zombie] <defunct>
```

Explain the result:

```
SID = 20235
 |
 ├── PGID 20235
```

```
|      └── bash
|
└── PGID 108752
       └── /zombie
```

This moment:

```
Foreground process group → PGID of bash
Background process group → PGID of ./zombie
```

2.4.5 Job Control

Run foreground `./zombie`
Then `Ctrl + Z`
It's job control. We are sending **SIGTSTP (Signal Terminal Stop)** to stopped `./zombie process`

So we can understand like that:

```
./zombie     → foreground job

./zombie &     → background job

Ctrl + Z        → stop foreground job

bg              → continue at background

fg              → return to foreground

Ctrl + C        → terminate foreground job
```

3. Memory layout + Linking + ELF

3.1 Memory layout

Example:

```
#include <stdio.h>
#include <stdlib.h>

int global_init = 10;    /* data */
int global_zero;        /* bss */

int main(void)
{
    int local = 20;         /* stack */

    int *p = malloc(sizeof(int));        /* heap */
    *p = 30;

    printf("text : %p\n", (void *)main);
    printf("data : %p\n", (void *)&global_init);
    printf("bss  : %p\n", (void *)&global_zero);
    printf("heap : %p\n", (void *)p);
    printf("stack: %p\n", (void *)&local);

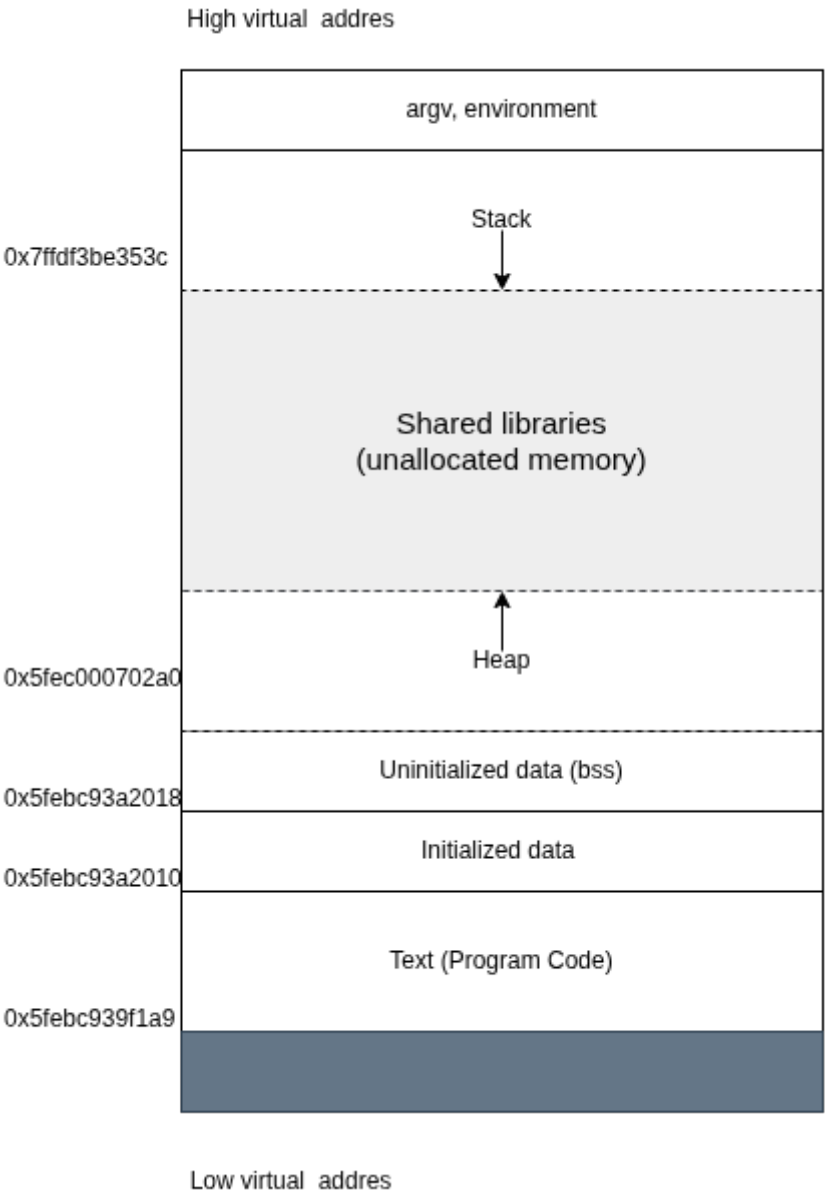
    free(p);

    return 0;
}
```

Result:

```
./memory_layout
text : 0x5febc939f1a9
data : 0x5febc93a2010
bss  : 0x5febc93a2018
heap : 0x5fec000702a0
stack: 0x7ffdf3be353c
```

We have typical **memory layout** of this process



- **text**:
  - contains the machine-language instructions of the program run by the process
  - **read_only** mode , so that a process doesn't accidentally modify due to pointer error
  - **sharable**, since many processes may be running the same program, so that single copy of the program code can be mapped into the virtual address space of all of the processes
- **data**:
  - ontains **global and static** variables that are explicitly initialized.
  - The values of these variables are read from the executable file when the program is loaded into memory
- **bss**:
  - contains **global and static** variables that are not explicitly initialized
  - Before starting the program, the system initializes all **memory** in this segment to **0**
  - **pros**: descrease **execute file size on disk**. They only need to record the location and size required for the uninitialized data segment, and this space is allocated by the program loader at run time.
- **heap**:

    - is an area from which memory (for variables) can be dynamically allocated at run time
    - The top end of the heap is called the **program break**.
- **stack**:
    - is the segment that automatically resize while call function call and return
    - One stack frame is allocated for each currently called function
    - A frame stores the function’s local variables (automatic variables), arguments, and return value

Lab 4

Run: `cat /proc/28160/maps` to check reality memory. (28160 is the PID of *memory_layout* program above)

```
64f33e95b000-64f33e95c000 r--p 00000000 103:06 2636329
/home/.../memory_layout
64f33e95c000-64f33e95d000 r-xp 00001000 103:06 2636329
/home/.../memory_layout
64f33e95d000-64f33e95e000 r--p 00002000 103:06 2636329
/home/.../memory_layout
64f33e95e000-64f33e95f000 r--p 00002000 103:06 2636329
/home/.../memory_layout
64f33e95f000-64f33e960000 rw-p 00003000 103:06 2636329
/home/.../memory_layout
64f3563d0000-64f3563f1000 rw-p 00000000 00:00 0                          [heap]
78e021000000-78e021028000 r--p 00000000 103:05 132783
/usr/lib/.../libc.so.6
78e021028000-78e0211bd000 r-xp 00028000 103:05 132783
/usr/lib/.../libc.so.6
78e0211bd000-78e021215000 r--p 001bd000 103:05 132783
/usr/lib/.../libc.so.6
78e021215000-78e021216000 ---p 00215000 103:05 132783
/usr/lib/.../libc.so.6
78e021216000-78e02121a000 r--p 00215000 103:05 132783
/usr/lib/.../libc.so.6
78e02121a000-78e02121c000 rw-p 00219000 103:05 132783
/usr/lib/.../libc.so.6
78e02121c000-78e021229000 rw-p 00000000 00:00 0
78e0213dc000-78e0213df000 rw-p 00000000 00:00 0
78e0213ef000-78e0213f1000 rw-p 00000000 00:00 0
78e0213f1000-78e0213f3000 r--p 00000000 103:05 132771                    /usr/.../ld-linux-
x86-64.so.2
```

```
78e0213f3000-78e02141d000 r-xp 00002000 103:05 132771                    /usr/.../ld-linux-
x86-64.so.2
78e02141d000-78e021428000 r--p 0002c000 103:05 132771                    /usr/.../ld-linux-
x86-64.so.2
78e021429000-78e02142b000 r--p 00037000 103:05 132771                    /usr/.../ld-linux-
x86-64.so.2
78e02142b000-78e02142d000 rw-p 00039000 103:05 132771                    /usr/.../ld-linux-
x86-64.so.2
7ffd02518000-7ffd0253a000 rw-p 00000000 00:00 0                          [stack]
7ffd025d2000-7ffd025d6000 r--p 00000000 00:00 0                          [vvar]
7ffd025d6000-7ffd025d8000 r-xp 00000000 00:00 0                          [vdso]
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0                  [vsyscall]
```

Explain the result:

```
Address range | Permissions | Offset | Device (Major:minor) | Inode | Pathname
Virtual address range | ↓  | Position start to map on disk | Identifier | Inode code | ...

r = read        w = write        p = private        x = execute

below heap is executable area
r-xp --> Here is the mapping area can EXECUTE --> main() function (.text segment)
rw-p --> Here is the mapping area can write (below HEAP) --> global variable (.data & .bss
segment)
[heap] --> is created by malloc()
[stack] --> is created by int local = 20;
between [heap] and [stack] is shared libraries.
Including:
  - libc.so.6 is mapped to virtual address space of process ( for printf() , malloc(), free()
)
  - ld-linux-x86-64.so.2 : dynamic linker/loader --> handle dynamic linking : load libc.so.6 +
resolve symbols + relocation
```

getlimit & setlimit function

The **getrlimit** can show the resource that each process can be used like the maximum of stack, text segment, file descriptors, bytes in size.

The **setrlimit** can set all field that we can see in the getrlimit function

Example:

```
int main()
{
   struct rlimit limit;
   getrlimit(RLIMIT_STACK, &limit);

   printf("soft limit of stack: %lu\n", limit.rlim_cur);
   printf("hard limit of stack: %lu\n", limit.rlim_max);

   printf("Stack soft limit: %.2f MB\n",
          (double)limit.rlim_cur / (1024 * 1024));

   printf("Stack hard limit: %.2f MB\n",
          (double)limit.rlim_max / (1024 * 1024));

   limit.rlim_cur = 16388608;



   setrlimit(RLIMIT_STACK, &limit);
   printf("soft limit of stack: %lu\n", limit.rlim_cur);
   printf("hard limit of stack: %lu\n", limit.rlim_max);

   printf("Stack soft limit: %.2f MB\n",
          (double)limit.rlim_cur / (1024 * 1024));

   printf("Stack hard limit: %.2f MB\n",
          (double)limit.rlim_max / (1024 * 1024));
}
```

Result:

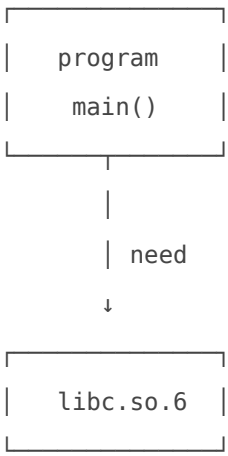
```
soft limit of stack: 8388608
hard limit of stack: 18446744073709551615
Stack soft limit: 8.00 MB
Stack hard limit: 17592186044416.00 MB
soft limit of stack: 16388608
hard limit of stack: 18446744073709551615
Stack soft limit: 15.63 MB
Stack hard limit: 17592186044416.00 MB
```

Soft limit is the maximum stack resource that this process can use.
Hard limit is the maximum satck resource that the system can allocate for this process.

3.2 Static and Dynamic Linking

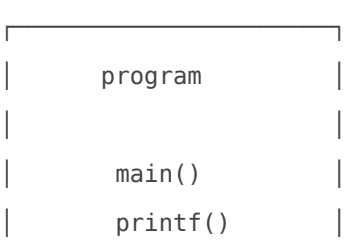
Feature	Static Link	Dynamic Link
Library format	Static Lib (.a - archive)	Share Lib (.so - shared object)
executation file size	Very big (contain all library code)	Very small (only contain dependency information)
RAM waste	wasted (each process compile once a copy)	save (many processes use one copy in RAM)
variable share	No share (each execute file is dependent)	Shared code (static/dynamic variable copy to each process)
execute speed	a little Fast	Slower at begin to positioning symbol
upgrade library	Compile and link all program/application	Only replace `.so` on system disk

Dynamic:



Library out of executable

Static:



```
|       libc code      |
|       ...            |
└──────────────────────┘
```

Necessary code of library in executable

3.3 ELF + nm + readelf + objdump

3.3.1 ELF (Executable and Linking Format)

ELF is format which Linux uses to perform many types of binary

```
┌──────────────────────────────┐
| ELF executable               | ← program run
├──────────────────────────────┤
| ELF object file (.o)         | ←  gcc -c result
├──────────────────────────────┤
| ELF shared library (.so)     | ← shared library
└──────────────────────────────┘
```

Example: `gcc test_elf.c -o elf_test` --> `elf_test` is a **ELF executable**
Run: `file elf_test`
Result:
`elf_test: ELF 64-bit LSB pie executable, x86-64, ...`

--> **ELF is the format of binary is created by compiler/linker, Linux loader understand it to run the program**

STRUCTURE

```
                  ELF FILE
┌──────────────────────────────────────┐
| ELF Header                           |
├──────────────────────────────────────┤
| Program Headers                      |
├──────────────────────────────────────┤
|                                      |
| Sections                             |
|   .text                              |
```

```
|   .rodata                             |
|   .data                               |
|   .bss                                |
|   ...                                 |
|                                       |
├───────────────────────────────────────┤
| Symbol Table                          |
├───────────────────────────────────────┤
| Relocation information                |
├───────────────────────────────────────┤
| ...                                   |
└───────────────────────────────────────┘
```

3.3.2 ELF header & Program Headers

ELF HEADER

Lab 5.1

ELF header is located in the beginning of file Example: `readelf -h elf_test`

```
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              DYN (Position-Independent Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x10e0
  Start of program headers:          ...
  Start of section headers:          ...
```

NOTE: `entry  point ≠ main()`

```
Linux kernel
     ↓
ELF loader
```

```
     ↓
entry point  /* first point when program running, point to _start function in C library
runtime crt1.o */
     ↓
C runtime startup
     ↓
main()
```

PROGRAM HEADER

Executable is loaded to **memory** by **loader**
We can use this command to check: `readelf -l elf_test`

3.3.3 Sections

```
Sections
├── .text
├── .rodata
├── .data
├── .bss
└── ...
```

Example:

```
int global_init = 10;
int global_zero;

const char *msg = "Hello";

int main()
{
    int local;
}
```

We can understand that:

```
C code
 │
 ├── machine code c□a main()
 │        ↓
```

```
 |      .text
 |
 ├── "Hello"
 |        ↓
 |      .rodata
 |
 ├── global_init = 10
 |        ↓
 |      .data
 |
 └── global_zero
          ↓
        .bss
```

text: contain *machine code*_ of program
Compile will convert it to machine instructions --> We can check by `objdump -d elf_test`

Lab 5.3 The output of `objdump` command

Example:

```
#include <stdio.h>
int global = 10;

int add(int a, int b)
{
    return a + b;
}
int main () {
    int sum = add(2,3);
    printf("%d\n", sum);
}
```

Run: `objdump -d test` Result (add function) :

```
0000000000001149 <add>:
    1149:       f3 0f 1e fa             endbr64
    114d:       55                      push   %rbp         /* Push BP to Stack */
    114e:       48 89 e5                mov    %rsp,%rbp    /* We have BP = SP of Stack Frame
of add() */
    1151:       89 7d fc                mov    %edi,-0x4(%rbp) /* 4 bytes (below of the bottom
```

```
of Stack)
                                                                (variable a) for edi register
*/
    1154:       89 75 f8                mov    %esi,-0x8(%rbp) /* 4 byte (below variable a)
for
                                                                esi register */
    1157:       8b 55 fc                mov    -0x4(%rbp),%edx /* push a to edx register */
    115a:       8b 45 f8                mov    -0x8(%rbp),%eax /* push b to eax register */
    115d:       01 d0                   add    %edx,%eax       /* eax = eax + edx | Caller
from main
                                                               will take result form eax
register */
    115f:       5d                      pop    %rbp            /* return BP to main() */
    1160:       c3                      ret                    /* return address */
/* 'e' register is low 32 bit of 'r' register 64 bit */
```

```
/* nm test */
We can said that nm test will show we the place of add() in memory, but objdump will show we
that in this address, What instruction CPU handle.
```

data: contain global/static variables is initialized with non-zero return value

bss: contain global/static variables is uninitialized

rodata(read only data):
Ex: `printf("Hello world");` --> "Hello world" is .rodata

NOTE: text/data/bss is not memory layout

- **text/data/bss in Section** is the way that organizing data by static file on disk (**Link-time view**) --> so that they can organize data in ELF
- **Memory layout** is virtual address space loaded and managed by OS in RAM while process running (**Execution-time review**)

Symbol Table

Ex:

```
#include <stdio.h>
int global = 10;


int add(int a, int b)
```

```
{
    return a + b;
}
int main () {
    int sum = add(2,3);
    printf("%d\n", sum);
}
```

Compiler/Linker need to know information about `global main add ...`

Lab 5.2

We can check it by `nm test`
Result:

```
000000000000038c r __abi_tag
0000000000001149 T add
0000000000004014 B __bss_start
0000000000004014 b completed.0
                 w __cxa_finalize@GLIBC_2.2.5
0000000000004000 D __data_start
0000000000004000 W data_start
0000000000001090 t deregister_tm_clones
0000000000001100 t __do_global_dtors_aux
0000000000003dc0 d __do_global_dtors_aux_fini_array_entry
0000000000004008 D __dso_handle
0000000000003dc8 d _DYNAMIC
0000000000004014 D _edata
0000000000004018 B _end
00000000000011a0 T _fini
0000000000001140 t frame_dummy
0000000000003db8 d __frame_dummy_init_array_entry
0000000000002110 r __FRAME_END__
.....................................
```

Char	Section	Mean
T / t	.text	
D / d	.data	
B / b	.bss	
R / r	.rodata	

Char	Section	Mean
U	Unidentify	Undefined - Need dynamic Linker find the shared lib in runtime
W / w	Everywhere	"Weak symbol - `_attribute_ weak variable`" If not define, it starts with default symbol (or NULL)

Explain the result

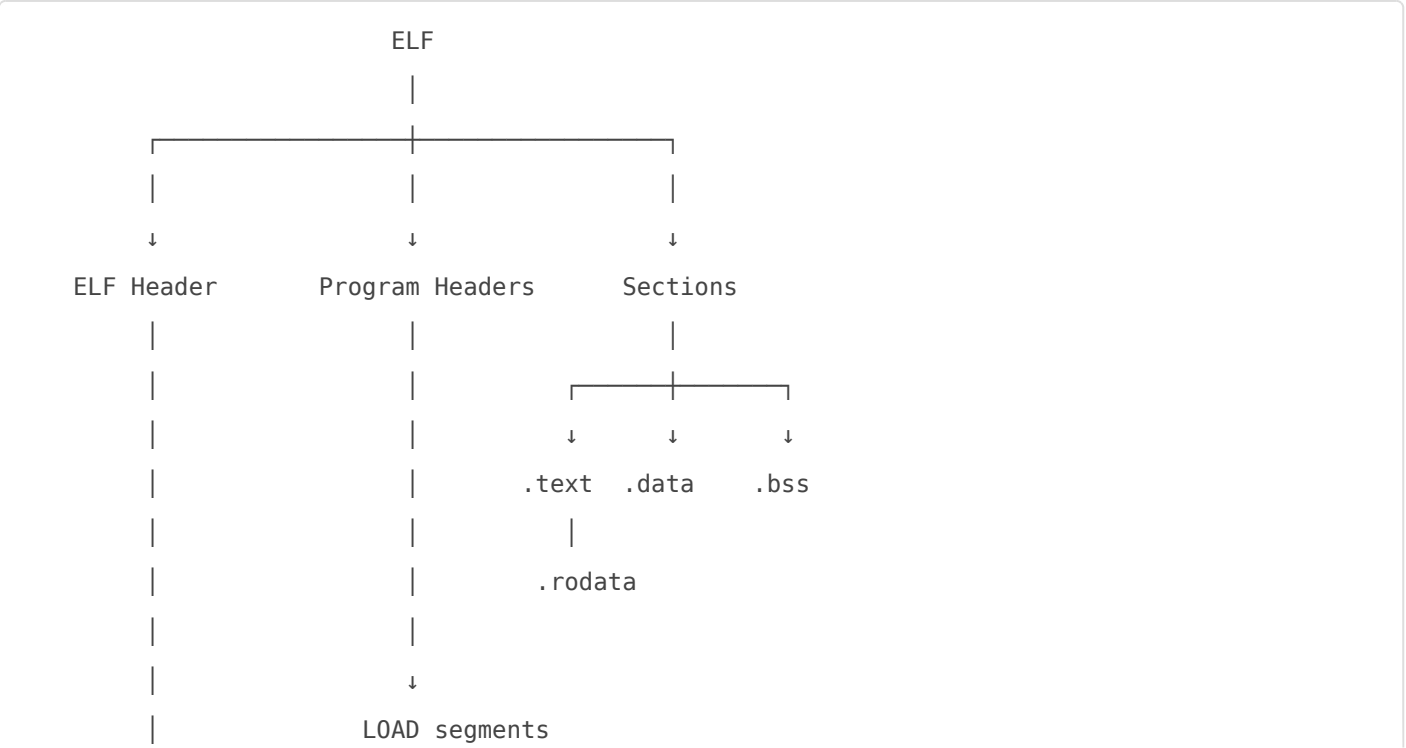
```
RULE:
  Upper letter: Global Symbol
  Lower letter: Local Symbol
```

This is the **necessary information** for **compiler/linker**
--> **Symbol table** contain informations about symbol like **function and variable** to **linker, loader, debugger and binary tools** can **identify** them

Relocation

For the example, we use `printf("Hello");`
but printf funtion belong to `libc.so.6`
So while build, program doesn't know exactly address of `printf`
--> Relocation will do `This reference need to fix (adjust) while link/load`

Summary of ELF



```
        │                 │
        │                 ↓
        │           Virtual Memory
        │
        ├── architecture
        ├── entry point
        └── file information

                  ┌─────────────────┐
                  │ Symbol Table    │
                  └─────────────────┘

                  ┌─────────────────┐
                  │ Relocation      │
                  └─────────────────┘
```

4. Assembly & Stack Frame

4.1 Assembly

Syntax

Instruction	Action	Example
mov	copy data	MOV %eax, %ebx --> ebx = eax
push	push data to stack	push AX
pop	get data out of stack	pop AX
add	add	ADD AX, BX
sub	sub	ADD AX, BX
lea	compute address	lea src_mem, dst_reg --> leaq 8(%rsp), %rax
call	call function	call printf@PLT
ret	return caller	ret
cmp	compare	cmp src, dst
jmp	jump	jmp .loop_start

Calculate the address. We have `offset(base, index, scale)`
--> address = base + (index * scale) + offset
Example:
leaq 8(%rsp), %rax --> rax = rsp + 8;
leaq 7(,%rdi,4), %rax --> rax = 0 + (rdi * 4) + 7
Scale only 1 2 4 8

4.2 Register

```
RAX   → return value
RDI   → argument 1
RSI   → argument 2
RDX   → argument 3
RCX   → argument 4
R8    → argument 5
R9    → argument 6
/* 6 beginning register for integer/pointer arguments */

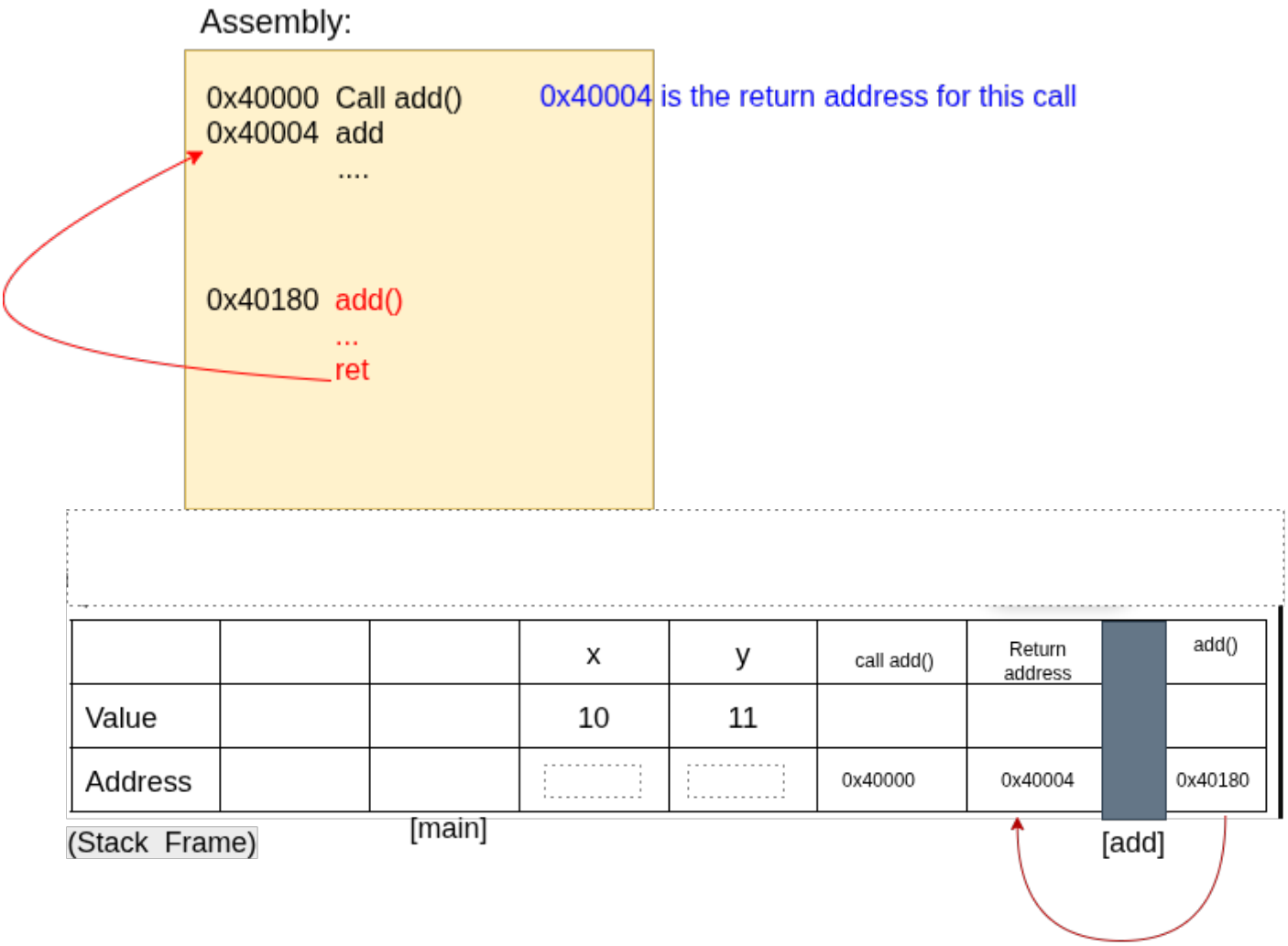
argument 7+ → stack

RSP   → stack pointer          /* top of Stack */
RBP   → frame/base pointer     /* fixed point in Stack memory*/
```

Return Address

Example:

```
#include <stdio.h>
int add(int a, int b) {
    return a + b;
}
int main() {
    int x = 10, y = 11;
    printf("Tong x + y = %d\n", add(x, y));
    Return 0;
}
```



4.3 Stack Frame

Stack Frame is created by [**gcc compiler** contact with **CPU** while execute machine code]

- Prologue (Create Frame):

```
pushq %rbp          # Save function's Base Pointer in front of (op top of) Stack
movq %rsp, %rbp     # Set new BP = top of Stack
subq $16, %rsp      # Move SP down for local variable space
```

- Epilogue (Delete Frame):

```
movp %rbp, %rsp    # Move SP to bottom of Frame
popq %rbp          # Pull BP out of Stack, Restore BP of previous function
ret                # get Return Address to back to caller
```

Question: Stack frame address is random? --> Both random and deterministic

- Randome base: Every time we re-run program, OS will choose random Stack Base to avoid attack of **Buffer Overflow**
- Deterministics Offsets: Stack Frame position rely on **Call Stack and SP** in this moment. Simple example is if `main()` call `add()`, `add()` force rightly below of `main()` (rely on **offset, variable size, data**)

4.4 Red Zone

leaf function: is the function which does not call another function

128 byte below RSP, we call Red Zone

If compiler know function is leaf function, it can use Red Zone for **temporary/local data**
dont need to `sub $16 , %rsp`
it can use directly this area:

```
RSP - 4
RSP - 8
...
```

dont need to change **RSP**

NOTE: WE MUST CAREFULLY use the function is **not leaf function**.
like

```
int calculate()
{
    foo();
    bar();
}
```

foo() and bar() have to call . This can be conflict the memory

Red Zone is the convention System V AMD64 ABI

Lab 6

Example:

```
#include <stdio.h>
int main() {
    int a = 1, b =2, c = 3, d = 4, e = 5, f = 6, g = 7;
```

```
    printf("total = %d", multi_argument(a,b,c,d,e,f,g));
}
int multi_argument(int a, int b, int c, int d, int e, int f, int g) {
    int x = a + b;
    int y = c + d;
    int z = e + f + g;
    int total = x + y + z;
    return total;
}
```

Run `gcc -O0 -fno-omit-frame-pointer -S multi_argument.c`
Result:

```
	.file	"multi_argument.c"
	.text
	.section	.rodata
.LC0:
	.string	"total = %d"
	.text
	.globl	main
	.type	main, @function
main:
.LFB0:
	.cfi_startproc
	endbr64
	pushq	%rbp           # Move BP into Stack
	.cfi_def_cfa_offset 16
	.cfi_offset 6, -16
	movq	%rsp, %rbp        # assign BP = SP
	.cfi_def_cfa_register 6
	subq	$32, %rsp        # Stack Allignment (multiple 16)
	movl	$1, -28(%rbp)    # push 1 to bottom of stack
	movl	$2, -24(%rbp)    # 2 above  1
	movl	$3, -20(%rbp)    # ...
	movl	$4, -16(%rbp)    # ...
	movl	$5, -12(%rbp)    # ...
	movl	$6, -8(%rbp)     # ...
	movl	$7, -4(%rbp)     # 7 above 6
	movl	-8(%rbp), %r9d   # assign regiter into address
	movl	-12(%rbp), %r8d
```

```
	movl	-16(%rbp), %ecx
	movl	-20(%rbp), %edx
	movl	-24(%rbp), %esi
	movl	-28(%rbp), %eax
	subq	$8, %rsp         # assign more 8 bytes (Stack) to Stack Frame can use
	movl	-4(%rbp), %edi   # push g = 7 to edi register
	pushq	%rdi             # push g to top of stack to transmit to multi_argument function

	movl	%eax, %edi       # push 1st argument a to edi; edi now is 32 bit register contain a --> g
	    # because multi_argument() receive 1st argument integer through %edi
	movl	$0, %eax        #set eax register = 0, not a = 0 --> main() prepare for call funtion
	    # we can understand that main() is setting state %eax before call
	call	multi_argument
	addq	$16, %rsp
	movl	%eax, %esi
	leaq	.LC0(%rip), %rax
	movq	%rax, %rdi
	movl	$0, %eax
	call	printf@PLT
	movl	$0, %eax
	leave
	.cfi_def_cfa 7, 8
	ret
	.cfi_endproc
.LFE0:
	.size	main, .-main
	.globl	multi_argument
	.type	multi_argument, @function
multi_argument:
.LFB1:
	.cfi_startproc
	endbr64
	pushq	%rbp
	.cfi_def_cfa_offset 16
	.cfi_offset 6, -16
	movq	%rsp, %rbp
	.cfi_def_cfa_register 6
	movl	%edi, -20(%rbp)
	movl	%esi, -24(%rbp)
	movl	%edx, -28(%rbp)
```

```
	movl	%ecx, -32(%rbp)
	movl	%r8d, -36(%rbp)
	movl	%r9d, -40(%rbp)
	movl	-20(%rbp), %edx
	movl	-24(%rbp), %eax
	addl	%edx, %eax
	movl	%eax, -16(%rbp)
	movl	-28(%rbp), %edx
	movl	-32(%rbp), %eax
	addl	%edx, %eax
	movl	%eax, -12(%rbp)
	movl	-36(%rbp), %edx
	movl	-40(%rbp), %eax
	addl	%eax, %edx
	movl	16(%rbp), %eax
	addl	%edx, %eax
	movl	%eax, -8(%rbp)
	movl	-16(%rbp), %edx
	movl	-12(%rbp), %eax
	addl	%eax, %edx
	movl	-8(%rbp), %eax
	addl	%edx, %eax
	movl	%eax, -4(%rbp)
	movl	-4(%rbp), %eax
	popq	%rbp
	.cfi_def_cfa 7, 8
	ret
	.cfi_endproc
.LFE1:
	.size	multi_argument, .-multi_argument
	.ident	"GCC: (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0"
	.section	.note.GNU-stack,"",@progbits
	.section	.note.gnu.property,"a"
	.align 8
	.long	1f - 0f
	.long	4f - 1f
	.long	5
0:
	.string	"GNU"
1:
```

```
.align 8
.long 0xc0000002
.long 3f - 2f
2:
.long 0x3
3:
.align 8
4:
```

Seminar Extra

1. After fork, heap memory is copied?

YES

> The two processes are executing the same program text, but they have separate copies of the stack, data, and heap segments. The child’s stack, data, and heap seg- ments are initially exact duplicates of the corresponding parts the parent’s memory. After the fork(), each process can modify the variables in its stack, data, and heap segments without affecting the other process.

```
static int idata = 111;                     /* Allocated in data segment */
int main()
{
    int istack = 222;                       /* Allocated in stack segment */
    int *p = malloc(sizeof(int));
    *p = 100;
    pid_t childPid;

    switch (childPid = fork()) {
    case -1:
        exit(1);
```

```
    case 0:
        idata *= 3;
        *p *= 3;
        istack *= 3;
        break;

    default:
        sleep(3);
        wait(NULL);
        break;
}
printf("PID=%d %s idata=%d p=%d istack=%d\n", getpid(),
(childPid == 0) ? "(child) " : "(parent)", idata, *p, istack);
free(p);
exit(0);
}
```

Result:

```
PID=7148 (child)  idata=333 p=300 istack=666
PID=7147 (parent) idata=111 p=100 istack=222
```

2. The limit of Process ID. Can we change this limit value?

With 32-bit platforms, the max value is 32768
With 64-bit, up to 2^22
The limit of Process ID is the value in the Linux-specific /proc/sys/kernel/pid_max file
Run: `cat /proc/sys/kernel/pid_max`
Result: `4194304` **WE CAN CHANGE THIS LIMIT VALUE!**
2 ways (Linux architecture)

1. Use `sudo sysctl -w kernel.pid_max=...`
2. Use this program to read /proc/sys/kernel/pid_max and change it

```
#include <stdio.h>
 #include <fcntl.h>
```

```
#include <string.h>
#include <stdlib.h>
#include <unistd.h>

#define MAX_LINE 100
int main(int argc, char *argv[])
{
    int fd;
    char line[MAX_LINE];
    size_t n;

    fd = open("/proc/sys/kernel/pid_max", (argc > 1) ? O_RDWR : O_RDONLY);
    if (fd == -1)
        exit(1);

    n = read(fd, line, MAX_LINE);
    if (n == -1)
        exit(1);

    if (argc > 1) {
        printf("Old value: %.*s\n", (int) n, line);

        if (lseek(fd, 0, SEEK_SET) == -1) {
                perror("lseek failed");
                close(fd);
                exit(1);
            }
        size_t len = strlen(argv[1]);
        if (write(fd, argv[1], len) == (ssize_t)len) {
            system("echo /proc/sys/kernel/pid_max now contains ""`cat
/proc/sys/kernel/pid_max`");
        }
        else perror("Write faile");
    }

 exit(0);
 }
```

Run: `sudo ./change_pid_max 12000`
Result:

```
cat /proc/sys/kernel/pid_max
12000
```

After this change, PID is reset if i PID > 12000:

```
PID=409 (child)  idata=333 p=300 istack=666
PID=408 (parent) idata=111 p=100 istack=222
```

> Once it has reached the **limit**, the process ID counter is reset to **300**, rather than 1. This is done because many low-numbered process IDs are in permanent use by system processes and daemons, and thus time would be wasted searching for an unused process ID in this range.

3. The purpose of Copy-on-write machine.

With Linux, we have the architecture of low-level:

```
Virtual Memory Space   ------------->  Stack - ... - Heap - Data - BSS - Text
        |
    Page Table
        |
Physical Memory Address
        |
       RAM
```

Page table will map Virtual Page (address) to Physical Page (address)
Ex: `x = 200`

```
virtual addr (of x): 0x77770000

Page table: map
Virtual Page              Physical Frame
0x77770000  ————————————→  0x3000A000
```

```
RAM:
0x3000A000
┌──────────────┐
│      200     │  ← x
└──────────────┘
```

> For the pages in the data, heap, and stack segments of the parent process, the kernel employs a technique known as copy-on-write

The different of old fork() and new `fork()` (use Copy-on-Write) Ex: `x = 200;  ->  fork();  -> x = 500;` (change x in parent)

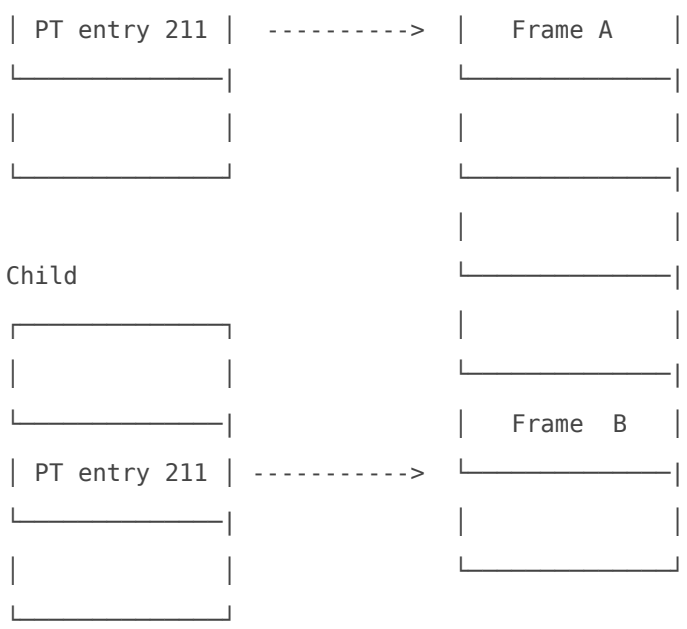
With fork(): Each process has its page table:

```
Parent
┌──────────────┐
│              │
└──────────────┘
│ PT entry 211 │
└──────────────┘
│              │
└──────────────┘


Child
┌──────────────┐
│              │
└──────────────┘
│ PT entry 211 │
└──────────────┘
│              │
└──────────────┘
```

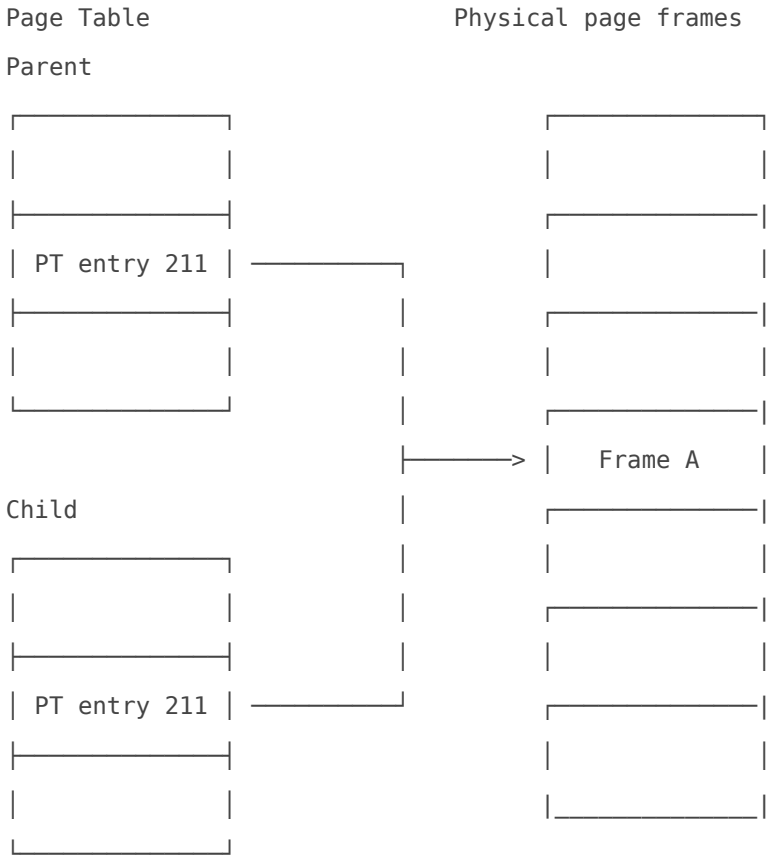
Old fork() will create 2 new **Physicall page frames** for 2 entries (immediately)

```
    Page Table              Physical page frames
Parent
┌──────────────┐              ┌──────────────┐
│              │              │              │
└──────────────┘|             └──────────────┘|
```

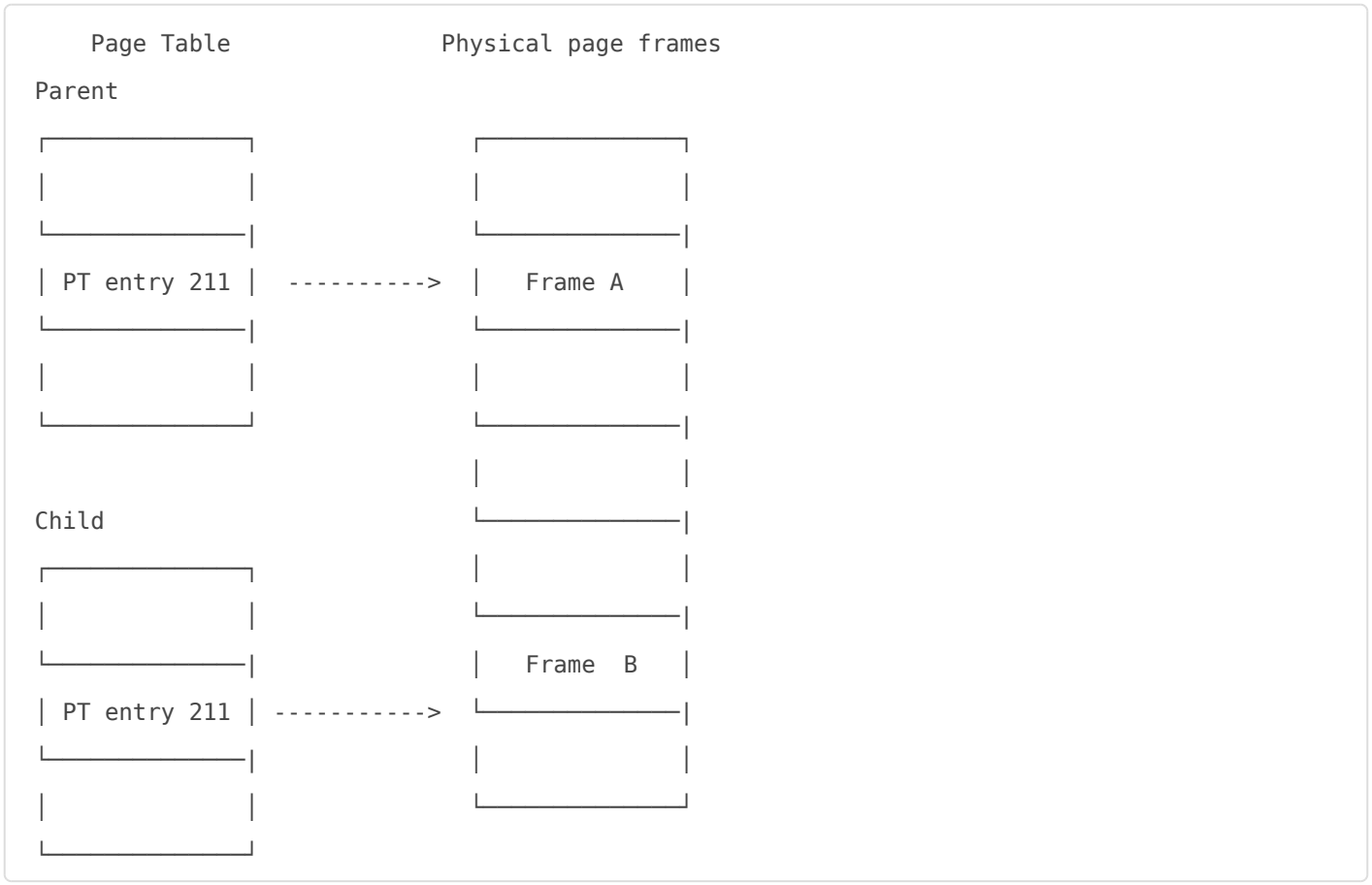


If we dont change anything about the value of x, it's still there in physical memory --> **waste RAM**
(It's dangerous if the data copy is so big , because double RAM so fast)
--> and also waste **copy time**

With modern `fork()`
After fork, before modification:



After modification:



4. What S+, Z+, R+ , T mean?

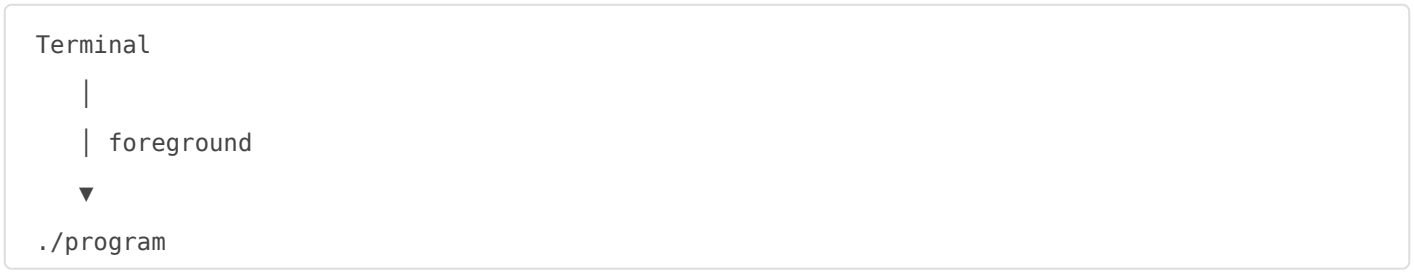
ps output

```
  PID    PPID    PGID     SID   TT       STAT   CMD
20235   11442   20235   20235 pts/3    S+  /usr/bin/bash --init-file /snap/co
108753  108752  108752   20235 pts/3    Z+   [zombie] <defunct>
```

separate STAT: **[PROCESS STATE][MODIFIER]**
PROCESS STATE: S = Sleep , R = Running, Z = Zombie, T = Terminate MODIFIER: `+`: process is belong to foreground process group of terminal `NOTHING`: process is not belong to foreground process group

We can running a process below background process authorize by `&` Ex : `./program &`
Shell is running job in background, return controlling authorize of terminal to shell
--> Shell now is foreground process group
--> ./program → background process group

If not have & Ex : `./program`
Shell give terminal to ./program -->



Revision #46
Created 2026-08-21 02:43:10 UTC by Lê Hoài Nam
Updated 2026-09-11 01:42:16 UTC by Lê Hoài Nam